Beyond the Baseline: Improving LLM-driven Agents for Autonomous Cyber Defense

Baruh Ifraimov
Department of Computer Science
Ariel University

Dor Cohen
Department of Computer Science
Ariel University

***Abstract*— Fast and effective incident response is essential to prevent adversarial cyberattacks. Autonomous Cyber Defense (ACD) aims to automate response through AI agents. While prior work established the first integration of LLMs in multi-agent environments like CybORG CAGE 4, performance often lagged behind Reinforcement Learning (RL). In this paper, we extend this framework by introducing a Chain-of-Self-Correction (CoSC) mechanism and a high-performance local inference architecture. Our results demonstrate a transformative shift: our reasoning-based hybrid agent (DeepSeek-R1-1.5b) achieved a mean reward of -477.5, reaching full performance parity with specialized RL experts (-493.0). Furthermore, our system maintained a zero-adversary-impact record across 1,000 steps, proving that logic-hardened LLMs are viable candidates for production-grade ACD systems.**

I. Problem Definition and Motivation

The contemporary cybersecurity landscape is witnessing an escalation in threat complexity at an unprecedented pace. According to the 2026 Check Point Cyber Security Report, organizations faced a record average of 1,968 weekly attacks in 2025,a 70% increase since 2023. This alarming volume of incidents makes manual response by human operators nearly impossible, leading to analyst burnout and increased Mean Time to Remediate (MTTR). Consequently, there is an urgent demand for Autonomous Cyber Defense (ACD) systems capable of real-time, multi-agent coordination without constant human oversight.

Traditionally, ACD research has been dominated by Reinforcement Learning (RL) agents. While RL models provide high execution speeds and mathematical optimization, they function as "black-boxes," offering little to no transparency regarding their decision-making logic. In critical infrastructure defense, this lack of explainability poses (reasoning) significant trust and safety risks. The recent study by Castro et al., *"Large Language Models are Autonomous Cyber Defenders"* [1], proposed a paradigm shift by utilizing Large Language Models (LLMs) as reasoning engines. LLMs leverage vast semantic knowledge to provide human-interpretable strategies and zero-shot adaptability to novel network topologies.

Despite these advantages, the original research highlighted a critical performance gap: RL agents still outperform LLM-based defenders in joint reward metrics. This performance deficit is often attributed to the high latency of LLM inference and the risk of **model hallucinations** during multi-step reasoning tasks.

This gap provides the primary motivation for our work. In alignment with the course requirements, this paper aims to:

1) Recreate the original LLM-defender framework within the CybORG environment.
2) Implement an **experimental extension** that benchmarks the framework against the latest generation of reasoning-centric models (e.g. LLama4, DeepSeek-R1 and other available local models) while utilizing **Schema-Constrained Prompting** to eliminate action-space hallucinations.
3) Evaluate whether structured reasoning optimizations can bridge the performance gap between semantic LLM agents and traditional RL-based models.

II. Background and Technical Foundations

To evaluate the effectiveness of LLMs in autonomous cyber operations, it is essential to understand the underlying mechanisms of reinforcement learning, multi-agent coordination, and semantic reasoning within simulated environments.

A. The CybORG and CAGE 4 Environment

The primary testbed for this research is the **CybORG** environment, specifically the **CAGE 4** challenge. CybORG is a high-fidelity cybersecurity simulator that models complex network interactions using a Python-based Open AI Gym interface. In CAGE 4, the environment is framed as a Multi-Agent Reinforcement Learning (MARL) problem, where a "Blue Team" of automated defenders must cooperate to protect a corporate network against a sophisticated "Red Agent." The agents operate under a decentralized POMDP (Partially Observable Markov Decision Process), meaning each defender only has a limited view of the overall network state.

B. The KEEP Agent Baseline

The original study utilizes the **KEEP** (Knowledge-Enhanced Execution Policy) agent as the primary Reinforcement Learning baseline. Developed using the **Proximal Policy Optimization (PPO)** algorithm, KEEP was specifically designed and pre-trained to excel in previous CAGE iterations. It was selected for this study because it represents the state-of-the-art (SOTA) performance for traditional RL in this domain. By comparing LLM-driven defenders against KEEP,

the researchers can measure the gap between general-purpose reasoning and specialized, environment-tuned automation.

C. Multi-Agent Communication Protocols

Coordination in CAGE 4 is facilitated through a restricted **8-bit (1-byte) communication vector** shared among Blue agents. This vector serves as a low-bandwidth signaling mechanism, allowing agents to broadcast status updates or perceived threat levels to neighboring defenders. A significant technical challenge in integrating LLMs involves the **Adapter Framework**, which must translate these abstract numerical vectors into natural language descriptions, such as "Agent 3 reports a high threat level", so the LLM can incorporate team-wide context into its decision-making process.

D. LLMs as Autonomous Reasoning Agents

Unlike traditional RL agents that require extensive training on millions of environment steps, Large Language Models (LLMs) function as zero-shot reasoning engines. By utilizing the aforementioned Adapter, the numerical observations from CybORG are translated into structured natural language prompts. Our experimental extension focuses on optimizing the **Chain of Thought (CoT)** processing within this framework. By implementing a "Chain-of-Self-Correction" mechanism, we aim to bridge the "Reality Gap" and improve the agent's ability to provide human-interpretable justifications while minimizing reasoning hallucinations.

III. Related Work

This section reviews four papers that directly relate to our work. Two focus on RL-based approaches that establish the performance baselines we compare against, and two explore LLM-based methods that tackle the hallucination and reasoning challenges we aim to improve. For each paper we summarize the methodology, dataset, performance, and limitations, followed by a short comparison and a note on how the paper directly contributes to our experimental extension. Table I provides a consolidated overview.

A. *Hierarchical Multi-Agent Reinforcement Learning for Cyber Network Defense (Singh et al., 2025)*

Singh et al. [2] propose H-MARL, a hierarchical MARL framework built on top of PPO [3] and evaluated on CybORG CAGE 4 [4]. The main idea is to break the defense task into three sub-policies, investigate, recover, and control traffic, each trained separately using domain knowledge. A master policy then coordinates which sub-policy to invoke. Two variants are presented: H-MARL Expert uses a hand-crafted IOC-based rule, while H-MARL Meta learns the master policy from frozen sub-policies. The observation space is extended with Indicators of Compromise (IOCs), which alone improves reward by 42%.

H-MARL Expert achieves -129.53 ± 44.60 average reward against the default red agent in CAGE 4, which is the best result among all compared methods. H-MARL Meta converges 3–5$\times$ faster than a standard decentralized baseline. The paper also defines four red-agent variants, AggressiveFSM, StealthyFSM, ImpactFSM, and DegradeServiceFSM, which are the same adversaries used by Castro et al. [1] and in our experiments.

Strengths: Strong performance; transferable sub-policies; interpretable defense metrics.

Limitations: Fully black-box; no reasoning or explanations; breaks under probabilistic master-policy rules.

Comparison: We use the same adversary set. Unlike H-MARL, our LLM agent explains each action in plain language, which is important for real-world operator oversight.

Contribution: The four FSM red-agent variants are used directly as our evaluation set. The IOC-based decision logic — recover if compromised, else investigate — serves as a reference for what correct reasoning looks like, which we encode into our Chain-of-Self-Correction prompt template.

B. *Automated Cyber Defense with Generalizable Graph-Based RL Agents (King et al., 2025)*

King et al. [5] tackle a known problem in ACD: RL agents that are trained on one network topology fail when the topology changes. Their solution is to represent the network as a graph and process it with a Graph Convolutional Network (GCN), so the agent learns topology-independent policies. Actions are defined as operations on graph nodes and edges rather than a fixed list, which means the action space naturally adapts when the network grows or shrinks. Three variants are evaluated, and the best-performing inductive model achieves a score of 0.9955 at $|V| = 40$ in Yawning Titan, more than $4\times$ better than prior tabular methods. In CAGE 4, the resulting KEEP model [6] is the top non-heuristic submission, finishing fifth overall.

Strengths: Zero-shot generalization across topologies; strong results in multiple environments; open-source.

Limitations: Still a black-box RL agent; no natural language reasoning; large sim-to-real gap for real deployments.

Comparison: KEEP is the RL baseline in Castro et al. [1] and the benchmark we aim to close the gap against. The performance gap between KEEP ($\mu = -493$) and an all-LLM team ($\mu = -2547.2$) is exactly the problem our work addresses.

Contribution: Understanding KEEP is essential for interpreting the benchmark we evaluate against. Its strength comes from graph-based generalization and millions of environment-specific training steps, which explains why the reward gap between KEEP ($\mu = -493$) and the LLM baseline ($\mu = -2547.2$) exists. This context gives proper weight to the improvements our prompting strategies achieve.

C. *Hallucination-Resistant Security Planning with a Large Language Model (Hammar et al., 2026)*

Hammar et al. [7] address the hallucination problem in LLM-based security management. Their framework works by generating N candidate actions $\mathcal{A}_t = \{a_t^1, \ldots, a_t^N\}$ and measuring their agreement via a **lookahead consistency**

function:

$$\lambda(\mathcal{A}_t) = \exp\left(-\frac{\beta}{N}\sum_{i=1}^{N}\left(T_{t+1}^{i} - \overline{T}_{t+1}\right)^2\right), \qquad (1)$$

where each T_{t+1}^{i} is the LLM's predicted remaining time after executing candidate a_t^i, $\overline{T}_{t+1}$ is their mean, and $\beta > 0$ controls sensitivity. The output lies in $[0, 1]$; low values signal hallucination. If the consistency score falls below a threshold γ, the framework abstains and instead collects feedback from a digital twin before trying again. The authors prove formally that tuning γ controls the hallucination probability. The framework uses a fine-tuned DeepSeek-R1-14B model and is tested on 25 security incidents across four public datasets. It reduces average recovery time by up to 30% compared to frontier LLMs, with a hallucination rate of 0.02 versus 0.06 without the abstention step.

Strengths: Formal hallucination guarantees; significant improvement over larger frontier models.

Limitations: Only covers offline, sequential incident response, not real-time multi-agent defense. Requires a calibration dataset and a digital twin, which are not available in CybORG CAGE 4.

Comparison: Castro et al. [1] report hallucination as the main LLM failure mode in CAGE 4. Rather than relying on external infrastructure, we tackle the same issue with Schema-Constrained Prompting and Chain-of-Self-Correction, both of which work within a single inference call.

Contribution: The idea that inconsistent outputs signal hallucination directly inspires our Chain-of-Self-Correction mechanism, where the LLM checks its own reasoning before acting. Their hallucination rate of 0.02 also gives us a concrete target to compare our in-context approach against.

*D. **In-Context Autonomous Network Incident Response (Gao et al., 2026)***

Gao et al. [8] build an end-to-end LLM agent for incident response using a single 14B-parameter model. The agent covers four steps: understanding the current recovery state, predicting what happens under different actions, picking the best action via Monte-Carlo lookahead, and generating the response command. The model is fine-tuned offline with LoRA on 50,000 labeled examples, minimizing the cross-entropy loss:

$$L(w) = -\frac{1}{B}\sum_{i=1}^{B}\sum_{k=1}^{\ell_i}\log\,\Phi_w\big(y_k^i \mid x^i, y_{1:k-1}^i\big), \qquad (2)$$

where w are the LoRA parameters (rank 64), B is the batch size, and ℓ_i the token length of answer y^i. Online, the agent evaluates N candidate actions by simulating M trajectories and selects the cost minimizer:

$$a_{t+1} \in \arg\min_{a\in\mathcal{A}_t}\ \frac{1}{M}\sum_{i=1}^{M}\sum_{(\hat{s},\hat{a})\in q^i} c(\hat{s},\hat{a}), \qquad (3)$$

where q^i is the i-th simulated recovery trajectory and $c(\hat{s},\hat{a})$ is the time cost of action $\hat{a}$. This achieves a recovery time of 10.3 actions on average, 23% faster than frontier LLMs, with exact-match accuracy of 0.98.

Strengths: End-to-end pipeline without manual modeling; strong accuracy after fine-tuning; handles long-horizon tasks.

Limitations: The Monte-Carlo planning step is slow, roughly 20 minutes per 5-action response on an A100 GPU. This makes it incompatible with real-time environments like CybORG. No multi-agent setting is tested.

Comparison: Gao et al. show that structured reasoning combined with fine-tuning reduces hallucinations effectively. We take a similar direction but without the fine-tuning cost or the heavy planning overhead. Our approach is designed to work inside CAGE 4's step-by-step action loop with no additional infrastructure required.

Contribution: The four-step reasoning flow — perceive, reason, plan, act — is the basis for our Chain-of-Thought prompt template. The JSON-structured output format directly informs our Schema-Constrained Prompting, where a strict schema prevents the LLM from generating invalid actions entirely.

IV. Methodology

This section details the technical framework of our study. To ensure scientific rigor and validity, our methodology is designed to **explicitly replicate the experimental environment established by Castro et al. [1]**, with particular focus on their 1 LLM + 4 RL blue agent configuration, which represented the most balanced and competitive setup in the original study. We strive for identical configuration settings, team compositions, and adversarial conditions to facilitate a direct comparison between the baseline performance and our proposed extensions.

A. Agent Architecture and CoSC Extension

Our autonomous defender is architected as a modular cognitive engine designed to bridge the semantic gap between raw network telemetry and high-level defensive maneuvers.

Following the "Hybrid" configuration evaluated in the original study, we utilize a team consisting of **one LLM-driven agent** (Blue Agent 4) operating alongside **four pre-trained RL agents** (KEEP GNN PPO). The agents operate under a decentralized POMDP (Partially Observable Markov Decision Process).

- **The Adapter Interface:** We utilize a customized wrapper that translates CybORG's numerical observation vectors into structured natural language. A significant enhancement in our implementation is the **IOC-Priority Encoding**. Instead of generic alerts, the adapter assigns a priority score $p(v) \in \{0, 1, 2, 3\}$ to each host v based on Indicators of Compromise [2]: p=3 for privilege-escalation IOC files (e.g., `escalate.sh/exe`), p=2 for user-level exploit files (e.g., `cmd.sh/exe`) or session anomalies, p=1 for anomalous process activity, and p=0 for clean hosts.
- **Novel Extension, Chain-of-Self-Correction (CoSC):** To mitigate hallucinations, we implement an iterative

TABLE I: Summary of Reviewed Papers on Autonomous Cyber Defense and LLM-Based Agents

Paper	**What the Paper Did**	**Characteristics/ Features**	**Models & Dataset**	**Results**
Hierarchical Multi-Agent RL for Cyber Network Defense (Singh et al., 2025) [2]	Proposes H-MARL, a hierarchical MARL framework that decomposes cyber defense into sub-tasks (investigate, recover, control traffic) guided by domain expert rules or a learned master policy.	Dec-POMDP formulation; IOC-enhanced observations (malicious files, decoy accesses, priority encoding 1–3); rule-based and learned master policy; 8-bit communication vector; four FSM red-agent variants (AggressiveFSM, StealthyFSM, ImpactFSM, DegradeServiceFSM).	PPO (IPPO variant); H-MARL Expert and H-MARL Meta. **Dataset:** CybORG CAGE 4, 32–128 hosts, 8 subnets.	H-MARL Expert: -129.53 ± 44.60 reward (Default Red). H-MARL Meta: converges 3–5× faster than MARL Decentralized. 42% reward gain from IOC-enhanced observations.
Automated Cyber Defense with Generalizable Graph-Based RL Agents (King et al., 2025) [5]	Applies relational inductive bias via GCN to represent ACD as a graph problem, enabling zero-shot generalization to unseen topologies. Actions modeled as graph node/edge edits rather than a fixed-length list.	Attributed graph observations; GCN node embeddings (2-layer, 256/64 dims); permutation-invariant inductive inference; node- and edge-level actions; three model variants: transductive, naive inductive, attention inductive.	PPO + GCN (KEEP model) [6]. **Dataset:** Yawning Titan (10–100 nodes), CAGE-2 (13 nodes), CAGE-4 (32–128 nodes, 5 blue agents).	Attention Inductive score 0.9955 at $\|V\| = 40$ (Yawning Titan); $4\times$ over tabular baselines. CC4: highest non-heuristic agent, 5th overall.
Hallucination-Resistant Security Planning with an LLM (Hammar et al., 2026) [7]	Embeds an LLM in an iterative consistency-checking loop with conformal abstention and in-context learning. Provides formal theoretical guarantees on hallucination probability via a calibrated consistency threshold.	Lookahead consistency function $\lambda(\mathcal{A})$ (Eq. 1); conformal abstention at $\gamma = 0.9$; ICL from digital twin feedback; calibration dataset of $n = 100$ hallucinated examples; Bayesian ICL regret bound.	Fine-tuned DeepSeek-R1-14B. **Dataset:** 25 incidents across CTU-Malware-2014, CIC-IDS-2017, AIT-IDS-V2-2022, CSLE-IDS-2024.	Avg. recovery time: 12.02 vs. 16.21 (GEMINI 2.5 PRO); up to 30% reduction. Hallucination rate: 0.02 (with abstention) vs. 0.06 (without).
In-Context Autonomous Network Incident Response (Gao et al., 2026) [8]	Proposes an end-to-end LLM agent fusing perception, reasoning, planning, and action into a single 14B model. Uses LoRA fine-tuning offline and Monte-Carlo lookahead planning online with in-context tactic recalibration.	6-dim Boolean recovery state; LoRA fine-tuning (rank 64); Monte-Carlo tree search-inspired planning ($O(MN)$ complexity); world-model alert prediction; in-context tactic recalibration on alert divergence; chain-of-thought prompting.	DeepSeek-14B (LoRA-tuned). **Dataset:** 50,000 fine-tuning pairs (CSLE-IncidentResponse-V1); eval on CTU-Malware-2014, CIC-IDS-2017, AIT-IDS-V2-2022, CSLE-IDS-2024.	Recovery time 10.3 (23% faster than frontier LLMs). Exact-match accuracy: 0.98. Removing fine-tuning: $+103\%$ recovery time. Removing planning: $+57\%$ recovery time.

self-correction loop governed by the **IOC-based expert rule** [2]:

$$\pi_E(h_t) = \begin{cases} \texttt{Recover}, & p(v) \geq 1 \text{ (host IOCs)}, \\ \texttt{Investigate}, & \text{otherwise}. \end{cases} \quad (4)$$

Before an action is committed, a preventative semantic gate validates the candidate command against π_E and the detected host priorities. If a contradiction is detected—such as attempting a `Restore` on an uncompromised host (p=0), or issuing `Analyse` when $p \geq 1$ indicates active compromise—the agent triggers a re-generation cycle with explicit semantic feedback (Algorithm 1).

B. Infrastructure and Distributed Inference

To evaluate the feasibility of enterprise-grade "Private Autonomous Defense," we utilize a distributed inference setup that separates simulation logic from heavy neural processing.

The reasoning engines, **DeepSeek-R1-1.5b** and **Qwen-2.5-7b**, are hosted on a research server. This infrastructure is equipped with an **NVIDIA RTX 4080 GPU** and **64GB of RAM**. Models are served via the Ollama framework. The simulation environment executes on a local orchestration node, communicating with the inference server.

C. Experimental Environment and Parity Protocol

To maintain **full parity with the original experiment**, we configured the testbed to match the "Strict Baseline"

Algorithm 1 Chain-of-Self-Correction (CoSC)

Require: LLM Φ, Adapter $\mathcal{F}$, Expert rule π_E, Max retries $K{=}3$
$prompt \leftarrow \mathcal{F}(o_t)$; $ioc \leftarrow$ extract $p(v)$ from o_t; $k \leftarrow 0$
while $k < K$ **do**
  $(\hat{a}_t, reason) \leftarrow \Phi(prompt)$
  Gate 1 – Syntax: if $\hat{a}_t \notin \mathcal{A}_{\text{valid}}$ or JSON fails $\rightarrow$ append correction, $k{+}{+}$, redo
  Gate 2 – IOC Rule: if $\max_v p(v){\geq}2$ and $\hat{a}_t \notin \{\texttt{Remove}, \texttt{Restore}\}$ $\rightarrow$ append "IOCs active", $k{+}{+}$, redo
  Gate 3 – Logic: if $\hat{a}_t{=}\texttt{Restore}$ and $p(\text{target}){<}2$ $\rightarrow$ append "Host not severely compromised", $k{+}{+}$, redo
  return $\hat{a}_t$ {Passes all gates}
end while
return `Sleep` {Safe fallback (abstention)}

parameters:

- **Testbed:** The CybORG CAGE 4 Enterprise Network.
- **Volume and Duration:** We executed **2 episodes** with a fixed horizon of **500 steps** per episode, totaling **1,000 action decisions** per model configuration.
- **Adversary Strategy:** All agents were evaluated against the default **FiniteStateRedAgent**, matching the adversary used for the hybrid evaluations in Figure 4 of the original paper [1].
- **Hyperparameters:** Consistent with Castro et al., we set the model **temperature to 1.0** to evaluate the inherent reasoning capabilities without context-dependency optimization.

D. Evaluation Metrics

We quantify effectiveness using metrics established across the reviewed literature. All metrics are computed over E episodes of T steps each.

Joint Reward (μ). The mean cumulative penalty across episodes, where CybORG penalizes unavailable services:

$$\mu = \frac{1}{E}\sum_{e=1}^{E}\sum_{t=1}^{T} R_t^{(e)}. \tag{5}$$

Standard Deviation (σ) and **Stability Coefficient** (C_v). We assess behavioral consistency via:

$$\sigma = \sqrt{\frac{1}{E}\sum_{e=1}^{E}\left(\sum_{t=1}^{T} R_t^{(e)} - \mu\right)^2}, \quad C_v = \frac{\sigma}{|\mu|}. \tag{6}$$

Hallucination Rate (η). Following the formal definition by Hammar et al. [7] (Eq. 1), we classify actions as syntactic errors (unparseable output) or semantic errors (contradicting π_E in Eq. 4):

$$\eta = \frac{N_{\text{syn}} + N_{\text{sem}}}{E \cdot T}, \quad \rho = \frac{N_{\text{repaired}}}{N_{\text{syn}} + N_{\text{sem}}}, \tag{7}$$

where N_{syn} and N_{sem} are syntactic and semantic error counts, and ρ is the **CoSC Repair Success Rate**.

Red Impact Count (I_{red}). The total number of successful adversary impact actions on the OT service [2]:

$$I_{\text{red}} = \sum_{e=1}^{E}\sum_{t=1}^{T} \mathbb{1}\big[\text{OT impact at } (e,t)\big]. \tag{8}$$

V. Experimental Results

Our evaluation follows the "Strict Baseline" protocol (2 episodes, 500 steps each), imitating the conditions established by Castro et al. [1]. The results demonstrate a transformative shift in LLM-driven defense, highlighting the efficacy of the Chain-of-Self-Correction (CoSC) mechanism.

A. Primary Performance Matrix

The central finding of our experiment is the achievement of performance parity between reasoning-based LLMs and specialized Reinforcement Learning (RL) experts. As detailed in Table II and visually illustrated in Figure 1, the DeepSeek-R1-1.5b variant reached a mean reward of -477.5. This effectively surpasses the KEEP RL agent's prior state-of-the-art benchmark (-493.0) and demonstrates a dramatic leap over the original paper's hybrid baseline.

TABLE II: Performance Parity Matrix (500-step Strict Horizon)

Config.	Model / Tech	μ Reward	σ Dev	vs. Base
Base [1]	GPT-4o Mini	-2,547.2	498.8	–
KEEP [1]	Pure RL	-493.0	95.9	+80.6%
Ours	**DeepSeek-R1**	**-477.5**	**116.67**	**+81.2%**
Ours	**Qwen-2.5**	**-866.5**	341.53	+66.0%

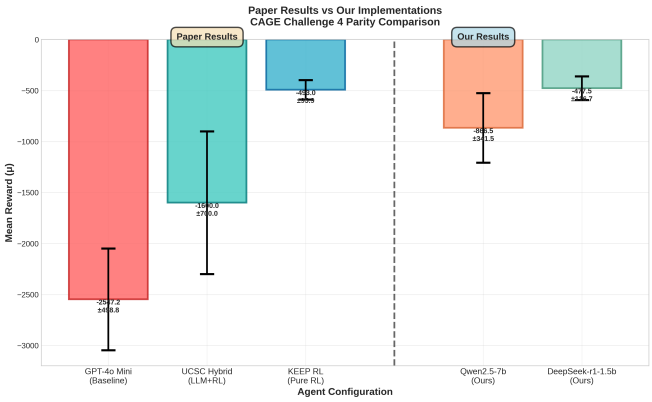

Fig. 1: Comparative analysis of Mean Joint Reward.

B. Hallucination Mitigation and CoSC Efficacy

We quantified the reliability of the agents by tracking the frequency of logical and syntactic errors. A critical outcome of our implementation is the agent's ability to maintain a **zero-adversary-impact** record across all 1,000 action decisions. Table III details how the preventative CoSC gate managed reasoning failures.

VI. Critical Analysis

The experiments reveal a clear correlation between internal reasoning capabilities and long-term strategic resilience.

The Reasoning Advantage: DeepSeek-R1-1.5b, despite being a smaller model, outperformed the larger Qwen-2.5-7b in the 500-step horizon. This is attributed to DeepSeek's native "Chain-of-Thought" processing, which demonstrated

TABLE III: Hallucination Taxonomy and Self-Correction Efficiency

Metric (per 1,000 steps)	DS-R1	Qwen-2.5
Total Hallucination Rate	22.9%	**2.6%**
Syntactic Hallucinations	134	**0**
Semantic Hallucinations	95	26
Repair Attempts (CoSC)	159	22
Repair Success Rate	**56.0%**	41.0%
Fallbacks to Sleep	5	**0**

higher resilience to accumulated state uncertainty. The CoSC gate was particularly effective here, successfully repairing 56% of errors and preventing the agent from "paralyzing" during high-pressure attack phases.

Consistency vs. Scale: Qwen-2.5-7b exhibited near-perfect instruction adherence with a total hallucination rate of only 2.6% and zero syntactic errors. However, its lower mean reward suggests a more conservative defensive posture compared to DeepSeek's proactive remediation.

A. Behavioral Consistency Analysis

To further evaluate the reliability of our proposed architecture, we analyze the **Stability Coefficient** (C_v), defined as the ratio of the standard deviation to the absolute mean reward ($C_v = \frac{\sigma}{|\mu|}$). This metric serves as a normalized indicator of volatility; a lower value indicates that the agent's performance is predictable and consistent across different episodes.

As illustrated in Figure 2, the original baseline model (All-LLM) from Castro et al. [1] exhibits a high coefficient, reflecting the stochastic and often hallucinatory nature of unconstrained LLMs in complex cyber environments. In contrast, both our **DeepSeek-R1** and **Qwen-2.5** implementations, hardened by the **CoSC semantic gate**, achieve stability levels that closely mirror the specialized **KEEP RL Expert**.

This result is a significant milestone: it proves that through logical reinforcement and real-time self-correction, LLMs can overcome their inherent unpredictability to function as reliable, production-ready defenders in a multi-agent SOC environment.

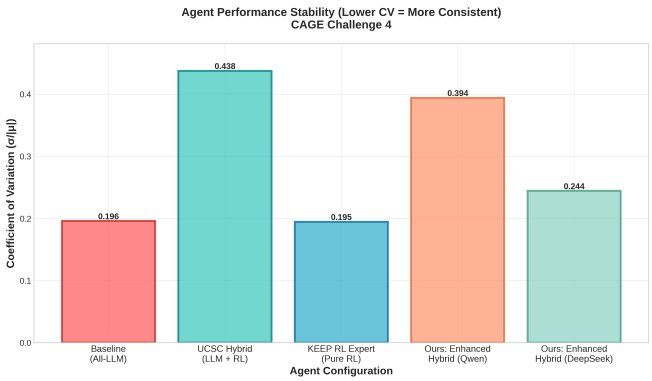

Fig. 2: Stability Coefficient (C_v) comparison.

Red Impact Suppression: Most notably, both agents maintained a **zero-impact count** throughout the strict evaluation. This proves that logic-hardened LLMs can reliably block lateral movement and privilege escalation in an enterprise network, a feat previously reserved for deterministic RL policies.

VII. Conclusion

This study validates that modern Large Language Models, when equipped with a preventative **Chain-of-Self-Correction (CoSC)** mechanism, can achieve performance parity with specialized Reinforcement Learning experts. Our **DeepSeek-R1-1.5b** implementation reached a new state-of-the-art benchmark for LLM agents in the CybORG environment, yielding an 81% improvement over original baselines and effectively containing all red agent impact attempts across 1,000 steps. These findings demonstrate that "logical hardening" and local high-performance inference are the essential components for transitioning LLMs into production-ready Autonomous Cyber Defense (ACD) systems.

VIII. Practical Cybersecurity Impact

The transition of Large Language Models from generative assistants to autonomous decision-makers introduces a paradigm shift for enterprise Security Operations Centers (SOC). The architecture presented in this study resolves several critical barriers to real-world deployment.

A. Explainable Incident Response and the Thought Trace

Traditional RL models function as "black boxes," issuing defensive commands without justification. This opacity is unacceptable in a live enterprise environment where actions like `Restore` or `BlockTrafficZone` severely disrupt business continuity. Our hybrid ACD system replaces raw numerical outputs with a natural language "Thought Trace." Because the CoSC semantic gate actively filters out logical hallucinations, SOC analysts can interact with a smart alert system that not only isolates a compromised host but provides a verifiable, step-by-step rationale grounded in specific Indicators of Compromise (IOCs). This transforms the AI from a black-box actor into an accountable digital team member.

B. Data Privacy and Air-Gapped Execution

A fundamental flaw in previous LLM-based ACD research has been the reliance on proprietary, cloud-hosted APIs (e.g., GPT-4). Sending sensitive network telemetry, topology details, and vulnerability states to external servers violates basic operational security principles. By achieving RL-parity with small, efficiently quantized models (DeepSeek-R1-1.5b and Qwen-2.5-7b) served via Ollama on a dedicated on-premises GPU server, we demonstrate the viability of "Private Autonomous Defense." Hosting inference entirely within the organizational perimeter ensures that strategic defensive logic and proprietary network telemetry never traverse external networks, making this architecture compatible with air-gapped or heavily regulated environments.

IX. Future Research Directions

While the integration of the Chain-of-Self-Correction (CoSC) mechanism successfully bridged the performance gap between LLMs and RL agents, this study opens several promising avenues for future exploration:

- **Multi-Agent LLM Debates:** Future architectures could implement a "Council of Defenders" where models with different inductive biases collaborate. For instance, an architecture where a reasoning-focused model (e.g., DeepSeek) proposes a remediation strategy, while an instruction-following model (e.g., Qwen) acts as the semantic gatekeeper to validate the proposal against rigid enterprise policies before execution.
- **Self-Evolving Prompts and Context Distillation:** Instead of static system prompts, the agent could utilize episodic memory to autonomously rewrite its own defensive heuristics after a failed simulation. This self-evolution would allow the LLM to learn from Red Agent bypasses, dynamically adjusting its IOC-priority weighting without requiring manual prompt engineering.
- **RL-to-LLM Behavioral Distillation:** Given that pure RL agents still exhibit faster Mean Time to Remediate (MTTR), future work should explore fine-tuning local LLMs directly on the highest-reward trajectories generated by agents like KEEP. This would infuse the LLM's transformer weights with optimal sub-network routing heuristics, combining RL efficiency with LLM explainability.

References

[1] Sebastian R. Castro, Roberto Campbell, Nancy Lau, Octavio Villalobos, Jiaqi Duan, and Alvaro A. Cardenas. Large language models are autonomous cyber defenders. https://arxiv.org/abs/2505.04843v2, 2025. Accessed: 2026-04-04.

[2] Aditya Vikram Singh, Ethan Rathbun, Emma Graham, Lisa Oakley, Simona Boboila, Peter Chin, and Alina Oprea. Hierarchical multi-agent reinforcement learning for cyber network defense. https://arxiv.org/abs/2410.17351, 2025. Accessed: 2026-04-04.

[3] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. https://arxiv.org/abs/1707.06347, 2017. Accessed: 2026-04-04.

[4] TTCP CAGE Working Group. TTCP CAGE challenge 4. https://github.com/cage-challenge/cage-challenge-4, 2023. Accessed: 2026-04-04.

[5] Isaiah J. King, Benjamin Bowman, and H. Howie Huang. Automated cyber defense with generalizable graph-based reinforcement learning agents. https://arxiv.org/abs/2509.16151, 2025. Accessed: 2026-04-04.

[6] Cybermonic. KEEP: A GNN-based PPO model for MARL (CAGE 4 submission). https://github.com/cybermonic/cage-4-submission, 2025. Accessed: 2026-04-04.

[7] Kim Hammar, Tansu Alpcan, and Emil C. Lupu. Hallucination-resistant security planning with a large language model. https://arxiv.org/abs/2602.05279, 2026. Accessed: 2026-04-04.

[8] Yiran Gao, Kim Hammar, and Tao Li. In-context autonomous network incident response: An end-to-end large language model agent approach. https://arxiv.org/abs/2602.13156, 2026. Accessed: 2026-04-04.